

# Nota pentru profesor

Material didactic: Labirintul Algoritmilor

Informatica | Clasa a IX-a | Metode de sortare

## 1. Descrierea materialului

"Labirintul Algoritmilor" este un joc educativ interactiv realizat in Python (tkinter) in care elevul trebuie sa ghideze pas cu pas doi algoritmi clasici de sortare pentru a "sorta cristalele" dintr-un labirint digital. Jocul are o poveste simpla care evolueaza in functie de raspunsurile elevului.

Jocul este format din doua camere (niveluri), fiecare dedicata unui algoritm diferit, si se ruleaza direct din terminal cu comanda:

```
python Labirintul_Algoritmilor.py
```

## 2. Ce exerseaza elevul

| Camera / Nivel             | Algoritm exersat — Ce face elevul                                                                                                                                       |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Camera 1 — Sala Bulelor    | Bubble Sort: la fiecare pas vede doua bare (elemente) iluminate si decide daca se schimba locurile (DA / NU). Raspuns corect = DA daca elementul din stanga e mai mare. |
| Camera 2 — Grota Minimului | Selection Sort: vede portiunea nesortata colorata si trebuie sa dea click direct pe bara cu valoarea cea mai mica (minimul). Portiunea sortata e marcata verde.         |

## 3. Competente specifice vizate (programa 2025)

| Cod CS | Descriere                                                                      |
|--------|--------------------------------------------------------------------------------|
| CS 3.2 | Aplica algoritmi specializati pe clase de probleme — sortare                   |
| CS 4.2 | Analizeaza comportamentul, avantajele si dezavantajele algoritmilor de sortare |
| CS 5.2 | Evalueaza corectitudinea si eficienta solutiilor cu algoritmi de sortare       |
| CS 6.2 | Proiecteaza solutii personalizate cu algoritmi de sortare in Python            |

## 4. Cum se poate folosi la clasa

- Activitate introductiva: elevii joaca jocul inainte de a invata teoria, pentru a descoperi singuri regulile algoritmilor
- Activitate de consolidare: dupa predarea Bubble Sort / Selection Sort, jocul fixeaza notiunile prin aplicare interactiva
- Tema facultativa: elevii pot rula jocul acasa si sa noteze scorul obtinut
- Punct de plecare pentru discutie: profesorul poate proiecta jocul si sa implice intreaga clasa in luarea deciziilor

## 5. Detalii tehnice

| Aspect          | Detaliu                                                      |
|-----------------|--------------------------------------------------------------|
| Limbaaj         | Python 3 (tkinter — inclus implicit in Python pe Windows)    |
| Fisier          | Labirintul_Algoritmilor.py                                   |
| Rulare          | python Labirintul_Algoritmilor.py (dublu-click sau terminal) |
| Array-uri       | Generate aleatoriu la fiecare joc — nu se repeta             |
| Sistem de vieti | 3 vieti per camera; o greseala = -1 viata                    |
| Scor            | 10 puncte / raspuns corect Camera 1; 15 puncte / Camera 2    |
| Durata medie    | 5-10 minute per joc complet                                  |

Nota: Array-urile sunt generate aleatoriu la fiecare rulare, deci elevii nu pot memora raspunsurile. Jocul poate fi jucat de mai multe ori cu configuratii diferite. Codul sursa este scris in Python cu comentarii in romana si poate fi analizat impreuna cu elevii ca exemplu de aplicatie cu interfata grafica (Tkinter) — continut CS 3.4 / 3.5 din programa.

## 6. Rezolvare în Python

```
# =====
# LABIRINTUL ALGORITMILOR
# Joc educativ interactiv – Informatica, clasa a IX-a
# Topic: Metode de sortare (Bubble Sort + Selection Sort)
# Tip: Aventura/Poveste cu animatii live
#
# Cum se joaca:
#   Camera 1 – Sala Bulelor (Bubble Sort):
#     La fiecare pas, doua cristale sunt comparate.
#     Tu decizi daca se schimba sau nu.
#   Camera 2 – Grota Minimului (Selection Sort):
#     Click pe cristalul minim din portiunea nesortata.
#   Ai 3 vieti per camera. Greselile le reduc.
#
# Rulare: python Labirintul_Algoritmilor.py
# =====
```

```
import tkinter as tk
import random
```

```
# -----
# PALETA DE CULORI (tema dark "digital fortress")
# -----
C = {
    "bg":          "#0D1117",
    "panel":       "#161B22",
    "border":      "#30363D",
    "text":        "#E6EDF3",
    "text_dim":    "#8B949E",
    "accent":      "#58A6FF",
```



```
"bar":      "#1F6FEB",
"bar_cmp":  "#E3B341",
"bar_swap": "#F85149",
"bar_ok":   "#3FB950",
"bar_min":  "#BC8CFF",
"bar_scan": "#FFA657",
"btn_yes":  "#238636",
"btn_no":   "#DA3633",
"btn_next": "#1F6FEB",
"btn_hover": "#30363D",
"gold":     "#E3B341",
"heart":    "#F85149",
}
```

## # TEXTELE POVEȘTII

```
POVESTE = {
    "intro": [
        "Ecranul se aprinde. Un mesaj apare:",
        "\"Bine ai venit, Algorist. Ai intrat in Labirintul Datelor.",
        "Cristalele sunt in dezordine. Haosul s-a instalat.",
        "Doar cel care stapaneste algoritmii de sortare poate",
        "restaura ordinea si deschide portile de iesire.\\"",
        "",
        "Doua camere te asteapta. Fiecare are cristale de sortat.",
        "Ai 3 vieti per camera. Nu le irosi.",
    ],
    "c1_intro": [
        "📍 CAMERA 1 – SALA BULELOR",
        "",
        "Peretii sunt acoperiti cu cristale care pulsează dezordonat.",
        "Vocea labirintului șopteste:",
        "\"Aplica Algoritmul Bulelor. La fiecare pas,",
        "doua cristale alaturate vor fi comparate.",
        "Tu decizi: se schimba locurile sau nu?\\"",
        "",
        "Regula: daca cristalul din stanga > cel din dreapta → SCHIMBA!",
    ],
    "c1_corect": [
        "Cristalele se reaseaza. Fluxul avanseaza...",
        "Exact! Labirintul vibreaza aprobator.",
        "Bine! Algoritmul merge pe drumul cel bun.",
        "Corect! Inca un pas catre ordine.",
        "Precis! Simti cum haosul cedeaza.",
    ],
    "c1_gresit": [
        "❌ Cristalul s-a crapat! Un ecou de eroare razbate...",
        "❌ Gresit! Datele se perturba. Ai pierdut o viata.",
        "❌ Eroare de flux! Labirintul te pedepseste.",
    ],
    "c1_win": [
        "🏆 SALA BULELOR – CUCERITA!",
        "",
    ],
}
```

```

        "Toate cristalele sunt ordonate. O usa de metal se",
        "deschide cu un zgomot adanc. Lumina albastra",
        "te ghideaza catre Camera a doua...",
    ],
    "c1_lose": [
        "💀 Ai epuizat toate vietile in Sala Bulelor.",
        "",
        "Cristalele raman in dezordine. Labirintul",
        "te readuce la inceput. Nu te descuraja -",
        "algoritmul iti da o noua sansa!",
    ],
    "c2_intro": [
        "📍 CAMERA 2 – GROTA MINIMULUI",
        "",
        "Cristalele sunt mai mari, mai grele. Vocea revine:",
        "\"Acum aplica Algoritmul Selectiei Minimului.",
        "La fiecare pas, portiunea NESORTATA apare in albastru.",
        "Gaseste cel mai MIC cristal si apasa pe el!\"",
        "",
        "Cristalele verzi sunt deja la locul lor. Nu le atinge.",
    ],
    "c2_corect": [
        "✅ Minimul gasit! Cristalul zboara la locul lui.",
        "✅ Exact! Selectia este corecta.",
        "✅ Bine! Portiunea sortata creste.",
        "✅ Precis! Grotă se lumineaza putin mai tare.",
    ],
    "c2_gresit": [
        "❌ Acela nu e minimul! Cristal gresit. -1 viata.",
        "❌ Eroare de selectie! Labirintul scartaie. -1 viata.",
        "❌ Nu! Cauta mai atent minimul.",
    ],
    "c2_win": [
        "🏆 LABIRINTUL ESTE CUCERIT!",
        "",
        "Un portal de lumina aurie apare. Cristalele sortate",
        "pulsează in armonie perfecta.",
        "",
        "\"Felicitari, Algorist! Ai dovedit ca stapanesti",
        "algoritmii de sortare. Labirintul iti deschide portile.\\"",
        "",
        "Ai esit victorios! 🎉",
    ],
    "c2_lose": [
        "💀 Camera 2 te-a invins. Dar stii mai mult acum.",
        "",
        "Regula: cauta intotdeauna cel mai MIC element",
        "din portiunea nesortata (albastrul).",
        "Incearca din nou – succesul e aproape!",
    ],
}

```

# FUNCTII HELPER – generare pasi algoritmi

```
def pasi_bubble(lista):  
    """Returneaza lista de pasi bubble sort.  
    Fiecare pas: (j, j+1, trebuie_swap, starea_listei_inainte)  
    """  
    pasi = []  
    arr = lista[:]   
    n = len(arr)  
    for i in range(n - 1):  
        for j in range(n - 1 - i):  
            trebuie = arr[j] > arr[j + 1]  
            pasi.append((j, j + 1, trebuie, arr[:], i))  
            if trebuie:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
    return pasi, arr
```

```
def pasi_selectie(lista):  
    """Returneaza pasi selection sort.  
    Fiecare pas: (i, poz_min, starea_listei_inainte)  
    """  
    pasi = []  
    arr = lista[:]   
    n = len(arr)  
    for i in range(n - 1):  
        poz_min = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[poz_min]:  
                poz_min = j  
        pasi.append((i, poz_min, arr[:]))  
        arr[i], arr[poz_min] = arr[poz_min], arr[i]  
    return pasi, arr
```

# CLASA PRINCIPALA – Jocul

```
class LabirintAlgoritmi:
```

```
    LATIME    = 780  
    INALTIME  = 610  
    N_BARE    = 6  
    MARJA     = 40  
    H_CANVAS  = 220
```

```
    def __init__(self):  
        self.root = tk.Tk()  
        self.root.title("🍀 Labirintul Algoritmilor – Informatica IX")  
        self.root.geometry(f"{self.LATIME}x{self.INALTIME}")  
        self.root.resizable(False, False)  
        self.root.configure(bg=C["bg"])
```

```

# Stare joc
self.vieti      = 3
self.scor       = 0
self.pasul_curent = 0
self.pasi       = []
self.array      = []
self.array_final = []
self.blocat     = False

self._build_ui()
self._ecran_intro()
self.root.mainloop()

```

## # CONSTRUCTIE UI

```

def _build_ui(self):

    hdr = tk.Frame(self.root, bg=C["panel"], pady=8)
    hdr.pack(fill="x")

    self.lbl_titlu = tk.Label(
        hdr, text="> LABIRINTUL ALGORITMIOR",
        font=("Consolas", 14, "bold"), bg=C["panel"], fg=C["accent"]
    )
    self.lbl_titlu.pack(side="left", padx=16)

    self.lbl_hud = tk.Label(
        hdr, text="",
        font=("Consolas", 12), bg=C["panel"], fg=C["text"]
    )
    self.lbl_hud.pack(side="right", padx=16)

    canvas_frame = tk.Frame(self.root, bg=C["border"], pady=1)
    canvas_frame.pack(fill="x", padx=12, pady=(10, 0))

    self.canvas = tk.Canvas(
        canvas_frame,
        width=self.LATIME - 24,
        height=self.H_CANVAS,
        bg=C["bg"], highlightthickness=0
    )
    self.canvas.pack()

    narr_frame = tk.Frame(self.root, bg=C["panel"], pady=8, padx=14)
    narr_frame.pack(fill="x", padx=12, pady=(6, 0))

    self.lbl_naratiune = tk.Label(
        narr_frame, text="",
        font=("Consolas", 10), bg=C["panel"], fg=C["text"],

```

```

        justify="left", anchor="w", wraplength=720
    )
    self.lbl_naratiune.pack(fill="x")

    self.lbl_intrebare = tk.Label(
        self.root, text="",
        font=("Consolas", 12, "bold"), bg=C["bg"], fg=C["gold"],
        wraplength=720
    )
    self.lbl_intrebare.pack(pady=(10, 6))

    self.btn_frame = tk.Frame(self.root, bg=C["bg"])
    self.btn_frame.pack(pady=4)

    self.btn_stanga = tk.Button(
        self.btn_frame, text="", width=22, pady=8,
        font=("Consolas", 11, "bold"),
        relief="flat", bd=0, cursor="hand2",
        command=self._click_stanga
    )
    self.btn_stanga.grid(row=0, column=0, padx=10)

    self.btn_dreapta = tk.Button(
        self.btn_frame, text="", width=22, pady=8,
        font=("Consolas", 11, "bold"),
        relief="flat", bd=0, cursor="hand2",
        command=self._click_dreapta
    )
    self.btn_dreapta.grid(row=0, column=1, padx=10)

    self.btn_continua = tk.Button(
        self.root, text="▶ Continua",
        font=("Consolas", 12, "bold"),
        bg=C["btn_next"], fg="white",
        activebackground="#1055B5",
        relief="flat", bd=0, padx=20, pady=8,
        cursor="hand2"
    )

```

#### # UTILITARE UI

```

def _set_hud(self):
    inimi = "♥" * self.vieti + "♥" * (3 - self.vieti)
    self.lbl_hud.config(text=f"{inimi}   Scor: {self.scor}")

def _set_naratiune(self, linii):
    """linii = lista de string-uri sau un singur string"""
    if isinstance(linii, list):
        text = "\n".join(linii)
    else:

```

```

        text = linii
        self.lbl_naratiune.config(text=text)

def _set_intrebare(self, text):
    self.lbl_intrebare.config(text=text)

def _ascunde_butoane(self):
    self.btn_stanga.grid_remove()
    self.btn_dreapta.grid_remove()

def _arata_butoane(self, txt_s, bg_s, txt_d, bg_d):
    self.btn_stanga.config(text=txt_s, bg=bg_s, fg="white",
                           activebackground=bg_s)
    self.btn_dreapta.config(text=txt_d, bg=bg_d, fg="white",
                           activebackground=bg_d)

    self.btn_stanga.grid()
    self.btn_dreapta.grid()

def _arata_buton_continua(self, callback):
    self._ascunde_butoane()
    self.btn_continua.config(command=callback)
    self.btn_continua.pack(pady=10)

def _ascunde_buton_continua(self):
    self.btn_continua.pack_forget()

# DESENARE BARE

def _deseneaza_bare(self, arr, culori_speciale=None, sorted_pana_la=-1):
    """
    Deseneaza barele pe canvas.
    culori_speciale: dict {index: culoare} pentru highlight-uri
    sorted_pana_la: indicele pana la care barele sunt sortate (verde)
    """
    if culori_speciale is None:
        culori_speciale = {}

    self.canvas.delete("all")
    n = len(arr)
    if n == 0:
        return

    val_max = max(arr) if max(arr) > 0 else 1
    cw = self.LATIME - 24
    latimea_bara = (cw - 2 * self.MARJA) / n
    spatiu = latimea_bara * 0.15
    bara_w = latimea_bara - spatiu
    h_util = self.H_CANVAS - 40 # spatiu sus pentru valori

    for i, val in enumerate(arr):
        x1 = self.MARJA + i * latimea_bara + spatiu / 2
        x2 = x1 + bara_w
        h_bara = int((val / val_max) * h_util)

```

```

y1 = self.H_CANVAS - 10 - h_bara
y2 = self.H_CANVAS - 10

# culoare
if i in culori_speciale:
    culoare = culori_speciale[i]
elif i <= sorted_pana_la:
    culoare = C["bar_ok"]
else:
    culoare = C["bar"]

# bara
self.canvas.create_rectangle(
    x1, y1, x2, y2,
    fill=culoare, outline=culoare, tags=f"bara_{i}"
)

# valoare deasupra
cx = (x1 + x2) / 2
self.canvas.create_text(
    cx, y1 - 12,
    text=str(val),
    font=("Consolas", 11, "bold"),
    fill=culoare if culoare == C["bar_ok"] else C["text"],
    tags=f"txt_{i}"
)

# index sub bara
self.canvas.create_text(
    cx, self.H_CANVAS - 3,
    text=f"[{i}]",
    font=("Consolas", 8),
    fill=C["text_dim"],
    tags=f"idx_{i}"
)

def _face_bare_clicabila(self, callback, exclude_verzi_pana_la=-1):
    """Leaga click-ul pe canvas la callback(index_bara)"""
    def _handler(event):
        if self.blocat:
            return
        # Calculam pe ce bara s-a dat click
        n = len(self.array)
        cw = self.LATIME - 24
        latimea_bara = (cw - 2 * self.MARJA) / n
        spatiu = latimea_bara * 0.15
        x = event.x
        for i in range(n):
            x1 = self.MARJA + i * latimea_bara + spatiu / 2
            x2 = x1 + latimea_bara - spatiu
            if x1 <= x <= x2 and i > exclude_verzi_pana_la:
                callback(i)
            return

```

```

        self.canvas.bind("<Button-1>", _handler)

def _dezleaga_bare(self):
    self.canvas.unbind("<Button-1>")

# ECRAN INTRO

def _ecran_intro(self):
    self.vieti = 3
    self.scor = 0
    self._set_hud()
    self._ascunde_buton_continua()
    self._ascunde_butoane()

    self._set_naratiune(POVESTE["intro"])
    self._set_intrebare("Esti gata sa intri in Labirint?")

    self._arata_butoane(
        "⚡ Da, intra in Labirint!", C["btn_yes"],
        "❌ Iesi din joc", C["btn_no"]
    )
    self._click_stanga_fn = self._start_camera1
    self._click_dreapta_fn = self.root.destroy

    self._animatie_intro()

def _animatie_intro(self):
    """Bare random pulsante pe ecranul de intro"""
    vals = [random.randint(2, 9) for _ in range(self.N_BARE)]
    cols = {i: random.choice([C["bar"], C["bar_cmp"], C["bar_swap"],
                             C["bar_ok"], C["bar_min"]])
            for i in range(self.N_BARE)}
    self._deseneaza_bare(vals, cols)

    self._intro_after = self.root.after(1200, self._animatie_intro)

def _stop_intro(self):
    if hasattr(self, "_intro_after"):
        self.root.after_cancel(self._intro_after)

# CAMERA 1 – BUBBLE SORT

def _start_camera1(self):
    self._stop_intro()
    self.vieti = 3
    self._set_hud()
    self._ascunde_butoane()
    self._ascunde_buton_continua()

    # Generam array

```

```

vals = random.sample(range(1, 10), self.N_BARE)
self.array = vals[:]
self.pasi, self.array_final = pasi_bubble(self.array)
self.pasul_curent = 0
self.sorted_pana_la_c1 = -1 # index pana la care s-a sortat

self._deseneaza_bare(self.array)
self._set_naratiune(POVESTE["c1_intro"])
self._set_intrebare("Apasa pe 'Incepe' cand esti gata!")
self._arata_butoane(
    "> Incepe Camera 1", C["btn_next"],
    "", C["btn_hover"]
)
self.btn_dreapta.config(state="disabled")
self._click_stanga_fn = self._pas_camera1
self._click_dreapta_fn = lambda: None

def _pas_camera1(self):
    """Arata pasul curent al Bubble Sort si pune intrebarea"""
    if self.pasul_curent >= len(self.pasi):
        self._finalizeaza_camera1(victorie=True)
        return

    self.blocat = False
    self._ascunde_buton_continua()

    pas = self.pasi[self.pasul_curent]
    j, j1, trebuie_swap, stare, trecere = pas
    self.array = stare[:]

    # Calculam sorted_pana_la: ultimele `trecere` elemente sunt sortate
    sorted_pana_la = self.N_BARE - 1 - trecere

    self._deseneaza_bare(
        self.array,
        culori_speciale={j: C["bar_cmp"], j1: C["bar_cmp"]},
        sorted_pana_la=sorted_pana_la - 1
    )

    # Intrebarea
    v_left = self.array[j]
    v_right = self.array[j1]
    self._set_intrebare(
        f"Cristalul [{j}]={{v_left}} vs Cristalul [{j1}]={{v_right}}\n"
        f"    {{v_left}} > {{v_right}} ? → Se schimba locurile?"
    )

    self._raspuns_corect_bubble = trebuie_swap

    self._arata_butoane(
        "< DA - Schimba!", C["btn_yes"],
        "> NU - Continua", C["btn_no"]
    )

```



```
self._click_stanga_fn = lambda: self._raspuns_camera1(True)
self._click_dreapta_fn = lambda: self._raspuns_camera1(False)

def _raspuns_camera1(self, raspuns_da):
    if self.blocat:
        return
    self.blocat = True

    pas = self.pasi[self.pasul_curent]
    j, j1, trebuie_swap, stare, trecere = pas
    sorted_pana_la = self.N_BARE - 1 - trecere

    self._ascunde_butoane()

    if raspuns_da == trebuie_swap:
        self.scor += 10
        self._set_hud()
        mesaj = random.choice(POVESTE["c1_corect"])

        if trebuie_swap:
            arr_nou = stare[:]
            arr_nou[j], arr_nou[j1] = arr_nou[j1], arr_nou[j]
            self._deseneaza_bare(arr_nou,
                                {j: C["bar_swap"], j1: C["bar_swap"]},
                                sorted_pana_la=sorted_pana_la - 1)
            self.root.after(400, lambda: self._dupa_animatie_c1(
                arr_nou, sorted_pana_la - 1, mesaj))
        else:
            # No swap - flash verde scurt
            arr_ok = stare[:]
            self._deseneaza_bare(arr_ok,
                                {j: C["bar_ok"], j1: C["bar_ok"]},
                                sorted_pana_la=sorted_pana_la - 1)
            self.root.after(350, lambda: self._dupa_animatie_c1(
                arr_ok, sorted_pana_la - 1, mesaj))
    else:
        # GRESIT
        self.vieti -= 1
        self._set_hud()
        mesaj = random.choice(POVESTE["c1_gresit"])

        arr_err = stare[:]
        self._deseneaza_bare(arr_err, {j: C["bar_swap"], j1:
C["bar_swap"]},
                                sorted_pana_la=sorted_pana_la - 1)
        self.root.after(500, lambda: self._dupa_animatie_c1(
            arr_err, sorted_pana_la - 1, mesaj))

        if self.vieti <= 0:
            self.root.after(700, lambda:
self._finalizeaza_camera1(victorie=False))
            return
```





```
self._set_naratiune(mesaj)

def _dupa_animatie_c1(self, arr, sorted_idx, mesaj):
    self._deseneaza_bare(arr, sorted_pana_la=sorted_idx)
    self._set_naratiune(mesaj)
    self.pasul_curent += 1
    self.blocat = False

    if self.pasul_curent >= len(self.pasi):
        self.root.after(600, lambda:
self._finalizeaza_camera1(victorie=True))
    else:
        self._arata_buton_continua(self._pas_camera1)

def _finalizeaza_camera1(self, victorie):
    self._ascunde_butoane()
    self._dezleaga_bare()

    if victorie:
        self._deseneaza_bare(self.array_final,
                               sorted_pana_la=self.N_BARE - 1)
        self._set_naratiune(POVESTE["c1_win"])
        self._set_intrebare("Camera 1 cucerita! ✅")
        self._arata_buton_continua(self._start_camera2)
    else:
        self._deseneaza_bare(self.array)
        self._set_naratiune(POVESTE["c1_lose"])
        self._set_intrebare("Game Over – Camera 1")
        self._arata_buton_continua(self._ecran_intro)

# CAMERA 2 – SELECTION SORT (click pe bare)

def _start_camera2(self):
    self._ascunde_buton_continua()
    self.vietii = 3
    self._set_hud()

    vals = random.sample(range(1, 10), self.N_BARE)
    self.array = vals[:]
    self.pasi, self.array_final = pasi_selectie(self.array)
    self.pasul_curent = 0

    self._deseneaza_bare(self.array)
    self._set_naratiune(POVESTE["c2_intro"])
    self._set_intrebare("Apasa pe 'Incepe' cand esti gata!")
    self._ascunde_butoane()
    self._arata_butoane(
        "▶ Incepe Camera 2", C["btn_next"], "", C["btn_hover"]
    )
    self.btn_dreapta.config(state="disabled")
    self._click_stanga_fn = self._pas_camera2

def _pas_camera2(self):
```



```

if self.pasul_curent >= len(self.pasi):
    self._finalizeaza_camera2(victorie=True)
    return

self.blocat = False
self._ascunde_butoane()
self._ascunde_buton_continua()

pas = self.pasi[self.pasul_curent]
i_start, poz_min, stare = pas
self.array = stare[:]

cols = {}
for k in range(i_start, self.N_BARE):
    cols[k] = C["bar"]

self._deseneaza_bare(
    self.array,
    culori_speciale=cols,
    sorted_pana_la=i_start - 1
)

self._set_intrebare(
    f"🔍 Gaseste MINIMUL din portiunea nesortata (indicii {i_start}-
{self.N_BARE-1})\n"
    f"    Click pe bara cu valoarea cea mai MICA!"
)
self._set_naratiune(
    f"Portiunea nesortata: {self.array[i_start:]}  "
    f"(sortata pana la [{i_start - 1}])"
    if i_start > 0 else
    f"Portiunea nesortata: {self.array}  (nimic sortat inca)"
)

# Activam click pe canvas
self._face_bare_clicabile(
    callback=self._raspuns_camera2,
    exclude_verzi_pana_la=i_start - 1
)

def _raspuns_camera2(self, idx_ales):
    if self.blocat:
        return
    self.blocat = True
    self._dezleaga_bare()

    pas = self.pasi[self.pasul_curent]
    i_start, poz_min, stare = pas
    arr = stare[:]

    if idx_ales == poz_min:
        # CORECT
        self.scor += 15

```

```

self._set_hud()
mesaj = random.choice(POVESTE["c2_corect"])

# Highlight minimul gasit (violet)
self._deseneaza_bare(arr,
    culori_speciale={idx_ales: C["bar_min"]},
    sorted_pana_la=i_start - 1)
self.root.after(500, lambda: self._efectueaza_swap_c2(
    arr, i_start, poz_min, mesaj))
else:
    # GRESIT
    self.vieti -= 1
    self._set_hud()
    mesaj = random.choice(POVESTE["c2_gresit"])
    # Flash rosu pe bara gresita
    self._deseneaza_bare(arr,
        culori_speciale={idx_ales: C["bar_swap"]}, poz_min:
C["bar_min"]},
        sorted_pana_la=i_start - 1)
    self._set_naratiune(mesaj)

    if self.vieti <= 0:
        self.root.after(700, lambda:
self._finalizeaza_camera2(victorie=False))
        return

    self.root.after(800, lambda: self._dupa_animatie_c2_gresit(
        arr, i_start, poz_min, mesaj))

def _efectueaza_swap_c2(self, arr, i_start, poz_min, mesaj):
    """Efectueaza swap-ul si actualizeaza display-ul"""
    arr_nou = arr[:]
    arr_nou[i_start], arr_nou[poz_min] = arr_nou[poz_min],
arr_nou[i_start]

    # Animam: ambele bare devin swap_color
    self._deseneaza_bare(arr_nou,
        culori_speciale={i_start: C["bar_swap"]},
        sorted_pana_la=i_start - 1)
    self.root.after(400, lambda: self._dupa_animatie_c2(arr_nou,
i_start, mesaj))

def _dupa_animatie_c2(self, arr, i_start, mesaj):
    # Acum i_start e sortat
    self._deseneaza_bare(arr, sorted_pana_la=i_start)
    self.array = arr[:]
    self._set_naratiune(mesaj)
    self.pasul_curent += 1
    self.blocat = False

    if self.pasul_curent >= len(self.pasi):
        self.root.after(700, lambda:
self._finalizeaza_camera2(victorie=True))

```

```

else:
    self._arata_buton_continua(self._pas_camera2)

def _dupa_animatie_c2_gresit(self, arr, i_start, poz_min, mesaj):
    """Dupa un raspuns gresit la Camera 2, asteptam si re-punem
    intrebarea"""
    self._deseneaza_bare(arr, sorted_pana_la=i_start - 1)
    self.blocat = False
    self._face_bare_clicabile(
        callback=self._raspuns_camera2,
        exclude_verzi_pana_la=i_start - 1
    )

def _finalizeaza_camera2(self, victorie):
    self._ascunde_butoane()
    self._dezleaga_bare()

    if victorie:
        self._deseneaza_bare(self.array_final,
                              sorted_pana_la=self.N_BARE - 1)
        self._set_naratiune(POVESTE["c2_win"])
        self._set_intrebare(f"🏆 Labirint cucerit! Scor final:
{self.scor}")
        self._arata_buton_continua(self._ecran_final_victorie)
    else:
        self._set_naratiune(POVESTE["c2_lose"])
        self._set_intrebare("Game Over – Camera 2")
        self._arata_buton_continua(self._ecran_intro)

# =====
# ECRAN FINAL VICTORIE
# =====

def _ecran_final_victorie(self):
    self._ascunde_buton_continua()

    # Animatie finala: bare se coloreaza in verde una cate una
    def coloreaza(idx):
        if idx < self.N_BARE:
            self._deseneaza_bare(self.array_final,
                                  sorted_pana_la=idx)
            self.root.after(180, lambda: coloreaza(idx + 1))

    coloreaza(0)

    self._set_naratiune([
        "★ FELICITARI! Ai terminat Labirintul Algoritmilor!",
        "",
        f"    Scor final: {self.scor} puncte",
        f"    Vieti ramase (Camera 2): {self.vieti}",
        "",
        "Ai demonstrat ca intelegi Bubble Sort si Selection Sort.",
        "Algoritmii nu mai au secrete pentru tine. 🎉",
    ])

```

```

    ])
    self._set_intrebare("Vrei sa joci din nou?")
    self._arata_butoane(
        "☺ Da, joaca din nou!", C["btn_yes"],
        "☒ Inchide jocul", C["btn_no"]
    )
    self._click_stanga_fn = self._ecran_intro
    self._click_dreapta_fn = self.root.destroy

# =====
# GESTIONARE CLICK-URI BUTOANE
# =====

def _click_stanga(self):
    if not self.blocat:
        self._click_stanga_fn()

def _click_dreapta(self):
    if not self.blocat:
        self._click_dreapta_fn()

if __name__ == "__main__":
    LabirintAlgoritmi()

```